

Dossier professionnel Bloc 4

Maintenir l'application logicielle en condition opérationnelle

Certification	Expert en développement logiciel : RNCP 39583
Bloc évalué	Bloc 4 : Maintenir l'application logicielle en condition opérationnelle
Projet support	Movie Picker : application web pour choisir un film à plusieurs
Candidat	Adrien MORAND
Date	Août 2026

Ce dossier décrit le maintien en condition opérationnelle d'une application **réellement déployée et utilisée** : les mesures, seuils, incidents et indicateurs qui y figurent proviennent de la production. Les mesures d'exploitation et les indicateurs d'usage ont été relevés fin juillet 2026, les retours utilisateurs recueillis à la mi-août.

§	Section	Compétence
1	Processus de mise à jour des dépendances	C4.1.1
2	Système de supervision et d'alerte	C4.1.2 (élim.)
3	Processus de collecte et de consignation des anomalies	C4.2.1 (élim.)
4	Fiche de consignation d'une anomalie	C4.2.1 (élim.)
5	Traitement d'une anomalie détectée en production	C4.2.2
6	Recommandations argumentées d'amélioration	C4.3.1
7	Journal des versions déployées	C4.3.2 (élim.)
8	Problème résolu en collaboration avec le support	C4.3.3

Contexte

Movie Picker répond à un irritant du quotidien : choisir un film à plusieurs sans négociation interminable. Un hôte crée une **soirée** et invite des participants par un lien ; chacun propose des films du catalogue TMDb, l'assemblée vote, puis une roue tire au sort parmi les propositions. L'application sert des utilisateurs réels depuis **avril 2026** à l'adresse **web.movie-picker.fr**, les comptes étant en production depuis le 8 avril et le périmètre s'étant complété par livraisons successives jusqu'à la **1.0.0**.

Composant	Technologie	Hébergement
Front (SPA installable en PWA)	React 19, TypeScript, Vite	AWS S3 + CloudFront
API REST	ASP.NET Core (.NET 10)	GCP Cloud Run (europe-west1)
Base de données	MongoDB	MongoDB Atlas

L'échelle réelle du service conditionne toutes les décisions présentées ici : **17 comptes inscrits**, **19 soirées créées** entre avril et juillet 2026, **15 161 requêtes** utilisateur sur les trente derniers jours. Cette taille ne justifie ni astreinte ni redondance multi-région ; elle justifie que la moindre indisponibilité soit détectée sans dépendre du signalement d'un utilisateur, et que chaque anomalie laisse une trace écrite. D'où un dispositif **proportionné**, mais complet sur la chaîne détection, consignation, correction, traçabilité.

§1. Processus de mise à jour des dépendances

Compétence C4.1.1 : Gérer les mises à jour des dépendances et des bibliothèques tierces, en surveillant régulièrement les nouvelles versions, en évaluant les impacts, et en les intégrant de manière sécurisée.

1.1 Périmètre logiciel

Le dépôt est un monorepo réunissant deux applications et leur outillage. Le code livré en production repose sur quatre écosystèmes, tous sous surveillance automatisée.

Écosystème	Manifeste	Contenu surveillé
npm / pnpm	pnpm-lock.yaml (racine + workspace apps/*)	Front React/TypeScript, outillage de build et de test
NuGet	apps/api-dotnet/**/* .csproj	API .NET, dépendances transitives incluses
GitHub Actions	.github/workflows/*.yaml	Actions du pipeline, épinglées par SHA de commit
Images Docker	apps/api-dotnet/MoviePicker.Api/Dockerfile	Images de base de l'API, épinglées par digest sha256

L'épinglage par SHA et par digest protège contre les attaques de chaîne d'approvisionnement ; en contrepartie ces références n'évoluent que si on les met à jour explicitement, d'où leur intégration au périmètre automatisé.

1.2 Fréquence

Trois rythmes se complètent, du plus lent au plus réactif.

Rythme	Mécanisme	Rôle
Mensuel	Dependabot : une pull request groupée par écosystème	Maintenir le socle à jour sans noyer le projet
Hebdomadaire	Analyse Trivy du dépôt entier (lundi 04 h 17 UTC), HIGH et CRITICAL bloquantes	Capter les vulnérabilités divulguées entre deux cycles
À chaque commit	Trivy sur pnpm-lock.yaml et dotnet list package --vulnerable --include-transitive	Interdire l'introduction d'une dépendance vulnérable et bloquer le déploiement

La cadence mensuelle groupée répond à une contrainte de projet mené par une seule personne : une pluie de demandes de fusion produit de la fatigue, puis des fusions non relues. Le filet de sécurité réel n'est pas la fréquence de l'outil de proposition, mais l'analyse hebdomadaire et l'audit à chaque commit, **bloquants** l'un et l'autre.

1.3 Type de mise à jour : automatique ou manuel

Étape	Automatique	Manuel
Surveillance des nouvelles versions, détection des vulnérabilités	✓	
Ouverture de la demande de fusion	✓	
Exécution des tests et portes de qualité	✓	
Évaluation d'impact et décision de fusion		✓
Montée de version majeure		✓
Déploiement après fusion	✓	

Aucune fusion n'est automatique. La proposition est automatisée, la décision ne l'est pas : une montée de version peut franchir toutes les portes tout en modifiant un comportement non couvert par les tests. Les demandes ouvertes par Dependabot s'exécutent d'ailleurs sans accès aux secrets du dépôt, ce qui désactive l'analyse de qualité externe et impose une relecture humaine. Chaque montée est jugée sur quatre points : la **nature du changement** (une majeure impose la lecture des notes de version), l'**exploitabilité réelle** de la vulnérabilité dans cette application, la **surface d'impact** et sa couverture de tests, et la **vérification** par les portes du pipeline, bloquantes, si bien qu'une régression détectable ne peut pas atteindre la production.

Le processus a été éprouvé le 25 juillet 2026 sur l'avis `GHSA-qwww-vcr4-c8h2` : `react-router` 7.18.1 est vulnérable et le déploiement se bloque de lui-même. L'avis ne concerne qu'un mode que cette SPA n'utilise pas, mais le correctif impose une montée majeure et un changement de paquet. Elle a été effectuée plutôt que neutralisée par une exception, l'interface utilisée étant stable et la couverture de tests forte : après réécriture des imports sur 50 fichiers et passage complet des portes, l'avis disparaît et le déploiement reprend, en moins d'une heure.

§2. Système de supervision et d'alerte

Compétence C4.1.2 (éliminatoire) : *Concevoir un système de supervision et d'alerte en déterminant le périmètre de supervision, en identifiant les indicateurs de suivi pertinents, en mettant en place des sondes et en configurant la modalité des signalements, afin de garantir une disponibilité permanente du logiciel.*

2.1 Périmètre de supervision

Les trois composants sont déployés séparément et peuvent tomber indépendamment : le périmètre les couvre tous les trois, ainsi que les dépendances externes dont l'indisponibilité dégrade le service.

Composant	Ce qui est surveillé
Front SPA (S3 + CloudFront)	Disponibilité de la page servie, erreurs JavaScript, Web Vitals
API .NET (Cloud Run)	Disponibilité, aptitude à servir, latence, taux d'erreur, exceptions serveur
Base MongoDB (Atlas)	Joignabilité depuis l'API
Services tiers (catalogue TMDb, courriel, notifications)	Indisponibilité, remontée en erreur serveur et capturée par le suivi des erreurs

La supervision de l'infrastructure sous-jacente relève des fournisseurs managés et n'est pas instrumentée par le projet : limite cohérente avec le choix d'un hébergement entièrement managé.

2.2 Indicateurs de suivi et critères de qualité

Les cibles ne sont pas des valeurs théoriques : elles ont été calées sur trente jours de mesures réelles relevées avant leur définition.

Indicateur	Mesure de référence	Cible
Disponibilité de l'API et du front	100 % sur les quatre semaines écoulées depuis la mise en service des sondes, le 25 juillet	≥ 99,5 % par mois
Latence p50 / p95 / p99	72 ms / 207 ms / 284 ms	< 200 ms / < 500 ms / < 1 s

Indicateur	Mesure de référence	Cible
Démarrage à froid	3,8 s	toléré (arbitrage de coût explicite)
Ping de la base depuis l'API	7 ms	< 100 ms
Taux d'erreur serveur	0,026 % (4 réponses 5xx sur 15 161 requêtes)	< 1 %
Erreurs applicatives	regroupées par empreinte	0 anomalie non triée au-delà de 24 h
Performance du front	médiane de trois exécutions	≥ 80, porte bloquante
Accessibilité du front	score 100 mesuré	≥ 98, porte bloquante

L'API tourne sans instance minimale : les seuils d'alerte tiennent compte du démarrage à froid plutôt que de le traiter comme une anomalie.

2.3 Sondes mises en place et finalité de chacune

Quatre familles se complètent, de la vérification externe la plus factuelle à la prévention en amont du déploiement ; les trois premières convergent vers le même signalement.

a. Sondes actives de disponibilité. Les seules qui prouvent qu'un utilisateur peut réellement atteindre le service : elles interrogent l'application **de l'extérieur**, depuis trois continents, indépendamment de son propre code.

Sonde	Cible	Fréquence	Finalité	Validation
Disponibilité API	api.movie-picker.fr/health	60 s	Le service répond-il ?	Code 2xx et status = ok
Aptitude à servir	api.movie-picker.fr/health/ready	15 min	Le service est-il opérationnel, base comprise ?	Code 2xx et dépendance mongodb à ok
Disponibilité front	web.movie-picker.fr/	5 min	La SPA est-elle servie par le CDN ?	Code 2xx et présence du titre attendu

Chaque sonde s'exécute depuis l'Europe, l'Amérique du Nord et l'Asie-Pacifique, délai d'expiration dix secondes : plusieurs régions évitent de confondre une panne réelle avec un incident réseau local.

Nom à afficher ↑	Asia Pacific	Europe	North America	South America	Règles
API - disponibilité (/health)	✓	✓	✓	●	1 ⋮
API - readiness (/health/ready, MongoDB)	✓	✓	✓	●	1 ⋮
Front - disponibilité (web.movie-picker.fr)	✓	✓	✓	●	1 ⋮

Les trois sondes de disponibilité et les régions depuis lesquelles elles interrogent le service

La distinction entre les deux sondes de l'API est le point central du dispositif. `GET /health` répond sans solliciter aucune dépendance : il détecte un service mort ou une révision qui ne démarre pas. `GET /health/ready` interroge la base avec un délai maximal de trois secondes et renvoie **503** si elle est injoignable : il détecte le cas, invisible pour la première sonde, où l'API répond parfaitement mais ne peut servir aucune donnée. Sa réponse porte aussi la **version déployée**. La même vérification est rejouée **en fin de déploiement** : une révision dont la base est injoignable fait échouer sa propre mise en production.

b. Sondes passives, métriques d'exécution. Collectées en continu par la plateforme d'hébergement, sans instrumentation applicative : requêtes par classe de code, distribution des latences, nombre d'instances actives. Leur finalité est de détecter les **dégradations progressives** que des sondes binaires « en ligne / hors

ligne » ne verraient jamais.

c. Sonde applicative, suivi des erreurs. Deux projets Sentry, un par composant, en production uniquement : exceptions front non gérées et toute réponse 5xx, y compris l'indisponibilité d'un service tiers, les erreurs métier attendues étant exclues pour éviter le bruit. Le partage se fait sur le code de réponse plutôt que sur le type d'exception, de sorte qu'une nouvelle erreur serveur est capturée sans qu'on ait à y penser. Chaque événement porte l'environnement, la **version déployée**, la route et le contexte ; les incidents sont regroupés par empreinte, avec compteur d'occurrences et version d'introduction. Sa finalité est de **nommer la cause** là où les sondes précédentes ne constatent qu'un symptôme.

d. Sondes préventives, avant la mise en production. Les portes du pipeline détaillées au §5.1, complétées par l'analyse de sécurité hebdomadaire. Leur finalité est de faire échouer le déploiement plutôt que l'utilisateur.

2.4 Seuils d'alerte

Cinq politiques sont configurées, dont celle des erreurs serveur qui porte deux conditions complémentaires. Leurs seuils sont volontairement placés **au-dessus du bruit mesuré** : une alerte qui se déclenche sans raison finit par être ignorée, et un système de supervision qu'on ignore ne supervise plus rien.

Politique	Condition	Seuil	Sévérité	Justification du seuil
API indisponible	Sonde <code>/health</code> en échec	≥ 2 points sur 5 min	Critique	Un seul point en échec traduit un incident réseau local
Base injoignable	Sonde <code>/health/ready</code> en échec	≥ 2 points sur 30 min	Critique	L'API répond mais ne peut servir aucune donnée
Front indisponible	Sonde front en échec	≥ 2 points sur 10 min	Critique	Même logique, sonde plus lente
Erreurs serveur anormales	Volume de réponses 5xx	> 2 sur 30 min	Erreur	Référence : 4 réponses 5xx en 30 jours
Erreurs serveur anormales	Part des réponses en 5xx	> 20 % pendant 10 min	Erreur	Capte la panne généralisée, quel que soit le volume
Latence dégradée	p95 des requêtes	> 800 ms pendant 10 min	Avertissement	Référence : 207 ms, marge pour les démarrages à froid

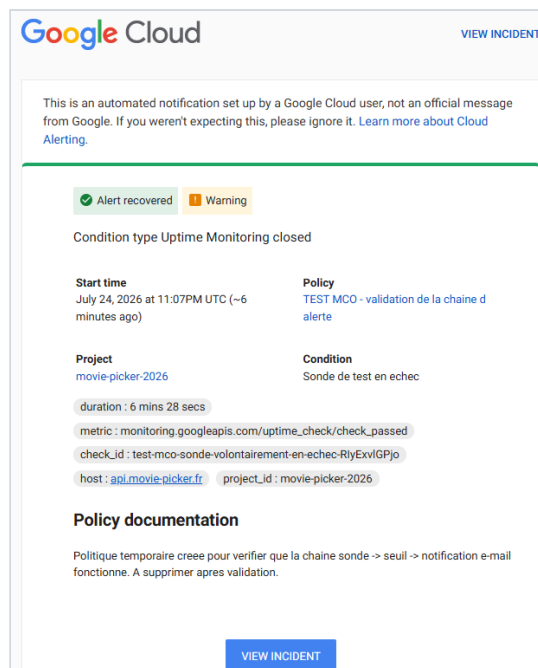
Cible et seuil d'alerte ne se confondent pas, ce que la latence illustre : la cible de 500 ms au p95 est un critère de qualité, suivie au tableau de bord et arbitrée à froid, tandis que l'alerte à 800 ms ne signale qu'une dégradation franche appelant une action immédiate. Une alerte calée sur la cible transformerait chaque démarrage à froid en incident.

Le doublement de la condition sur les erreurs serveur résulte d'un calcul de volume. Celle en **volume absolu** travaille au quotidien : elle détecte le cas le plus courant, une partie des routes applicatives qui tombe alors que le service continue de répondre. Celle en **part du trafic** couvre la panne généralisée, mais les sondes la diluent mécaniquement, ajoutant une trentaine d'appels toujours en 200 par fenêtre de dix minutes contre trois à quatre requêtes utilisateur : une défaillance limitée aux routes applicatives ne franchirait pas les 20 %, ce que la condition en volume rattrape. Chaque politique embarque sa **conduite à tenir**, affichée dans la notification.

2.5 Modalité de signalement

Le signalement se fait par **courriel** vers l'adresse d'exploitation, déclarée comme canal de notification et rattachée aux cinq politiques. Le message porte la politique déclenchée, la condition franchie, la ressource concernée, l'horodatage et la durée de l'incident, un lien vers celui-ci et la conduite à tenir. Trois règles par projet Sentry couvrent les erreurs applicatives : anomalie prioritaire, **régession** d'une anomalie corrigée, **rafale** de plus de vingt occurrences en une heure ; la deuxième protège contre une correction qui se défait silencieusement plusieurs déploiements plus tard.

La chaîne complète a été **vérifiée de bout en bout** : une sonde temporaire pointant vers une adresse inexistante a été mise en service, l'alerte s'est déclenchée après deux points de contrôle en échec, le courriel est parvenu à l'exploitant avec sa conduite à tenir, puis l'incident s'est refermé au retour à la normale. La sonde et la politique de test ont ensuite été supprimées : le dispositif n'est pas seulement configuré, il est prouvé.



Notification reçue lors de la vérification de la chaîne de signalement : politique de test, condition franchie, ressource visée, durée de l'incident et conduite à tenir, avec clôture automatique au retour à la normale

Le projet étant exploité par une seule personne, il n'y a ni astreinte ni escalade. Toute alerte débouche sur une action, qualifiée à partir de l'incident puis consignée en fiche écrite selon le processus du §3.

2.6 Tableau de bord, coûts et limites

Un tableau de bord d'exploitation regroupe les six vues suivies au quotidien : disponibilité de l'API, disponibilité du front, latences p50 et p95 avec le seuil d'alerte matérialisé, requêtes par classe de code, erreurs serveur avec son seuil de volume, et nombre d'instances actives. C'est l'écran consulté en premier lors d'un signalement, avant d'ouvrir l'incident lui-même.

Le dispositif entier tient dans les offres gratuites, les sondes ajoutant environ 138 000 requêtes mensuelles à l'API, soit 7 % du quota offert. Effet de bord favorable : cette interrogation permanente maintient une instance tiède aux heures d'usage, ce qui raréfie pour l'utilisateur les démarrages à froid que le §2.2 tolère. Deux limites sont assumées : la rétention des erreurs est d'environ trente jours, ce qui impose d'archiver hors outil les incidents à conserver, et la base de données n'est supervisée que dans sa joignabilité depuis l'API.

§3. Processus de collecte et de consignation des anomalies

Compétence C4.2.1 (éliminatoire) : Consigner les anomalies détectées en élaborant un processus de collecte et de consignation, en utilisant des outils de collecte et en y intégrant toutes les informations pertinentes, afin de déterminer le correctif à mettre en place.

3.1 Principe

Une anomalie confirmée donne lieu à une fiche écrite. Le gestionnaire d'incidents du dépôt (GitHub Issues) est la source unique de vérité : aucune correction n'est déployée sans une trace expliquant ce qui ne va pas, comment le reproduire et ce qui a été décidé. Sur un projet mené par une seule personne, la règle est à la fois la plus fragile et la plus utile : trois semaines après, la cause racine n'est plus reconstituable de tête. Le processus est dimensionné pour la typologie du logiciel, une application web grand public en déploiement continu, sans astreinte, dont les anomalies proviennent presque toutes de trois origines : une régression introduite par un déploiement, une dépendance externe défaillante, ou un cas d'usage non prévu.

3.2 Canaux de collecte

Cinq canaux alimentent le processus, du plus automatique au plus humain.

#	Canal	Ce qu'il détecte	Délai
1	Supervision : 3 sondes, 5 politiques (§2)	Indisponibilité, base injoignable, erreurs serveur, latence dégradée	2 à 30 min
2	Suivi des erreurs : 2 projets, 3 règles chacun (§2)	Exception front ou serveur, régression, rafale	Immédiat
3	Retours utilisateurs : lien « Signaler un problème »	Anomalie fonctionnelle sans exception : comportement inattendu, parcours bloqué	Variable
4	Portes du pipeline : tests, performance, accessibilité, qualité, vulnérabilités, secrets	Régression, vulnérabilité, dégradation, avant la production	Chaque commit
5	Recette manuelle	Écart avec le cahier de recettes	Chaque évolution

Les canaux 1, 2 et 4 sont automatiques et notifient par courriel. Le canal 3 est le seul qui dépende d'une démarche humaine, et c'est celui qui remonte les anomalies les plus coûteuses, puisqu'elles ont déjà atteint l'utilisateur sans déclencher d'alerte technique : l'anomalie du §4 en est l'illustration, l'application répondant normalement pendant que les utilisateurs perdaient leur session. C'est pour réduire la dépendance à ce canal que la supervision a été renforcée, et pour le rendre plus efficace que le lien de signalement a été ajouté.

3.3 Outil et gabarit de consignation

Les fiches vierges sont **désactivées** : tout signalement passe par un formulaire structuré, ce qui garantit que les informations nécessaires à la reproduction sont présentes dès la création.

Champ	Obligatoire	Pourquoi il est nécessaire
Contexte (URL, navigateur, système, version)	oui	Sans lui on corrige à l'aveugle un défaut peut-être propre à un environnement
Étapes pour reproduire	oui	Une anomalie non reproductible n'est ni corrigible avec certitude, ni vérifiable
Comportement attendu	oui	Distingue le défaut réel du malentendu fonctionnel
Comportement observé	oui	Décrit le symptôme indépendamment de son interprétation

Champ	Obligatoire	Pourquoi il est nécessaire
Captures ou journaux	non	Raccourcit fortement le diagnostic
Sévérité	oui	Détermine le délai de prise en charge

Les fiches portent des étiquettes normalisées : `bug`, `triage` (posée automatiquement à la création), `severite:critical|high|medium|low`, `production` lorsque l'anomalie est constatée en production, et `regression` lorsqu'il s'agit de la réapparition d'une anomalie déjà corrigée.

3.4 Qualification et délais de prise en charge

La sévérité est déclarée par le signalant puis confirmée au triage. Elle mesure l'**impact utilisateur**, jamais la difficulté technique : une anomalie triviale peut bloquer tout le monde, une correction complexe n'affecter personne.

Sévérité	Définition	Prise en charge	Correction visée
critical	Service inutilisable ou perte de données	Immédiate, tout autre travail suspendu	Sous 24 h, ou retour arrière immédiat
high	Fonctionnalité principale bloquée, sans contournement	Sous 24 h	Sous 72 h
medium	Gêne réelle mais contournement possible	Au prochain lot de travail	Prochaine version mineure
low	Cosmétique ou marginal	Mise en file	Sans engagement de date

Une anomalie critique ouvre un arbitrage immédiat, **corriger** ou **revenir en arrière**. Le retour arrière est privilégié lorsque l'anomalie suit un déploiement : il rétablit le service en quelques minutes sans exiger d'avoir compris la cause.

3.5 Cycle de vie d'une anomalie

Étape	Contenu
1. Signalement	Fiche créée via le gabarit, quel que soit le canal. Une alerte ou une erreur capturée est recopiée avec le lien vers l'événement d'origine
2. Triage	Sévérité confirmée, étiquette <code>triage</code> retirée, <code>production</code> ou <code>regression</code> ajoutée le cas échéant
3. Reproduction	Rejeu des étapes déclarées. Une anomalie non reproductible reste ouverte, documentée avec ce qui a été tenté, tant que le signalement persiste
4. Analyse	Cause racine identifiée et consignée. Symptôme et cause distingués, une même déconnexion pouvant venir d'un cookie, d'une clé ou d'une configuration inopérante
5. Correction	Branche dédiée, test de non-régression lorsque la nature du défaut le permet
6. Vérification	Validation en production, par le contrôle automatique en fin de déploiement puis vérification du comportement d'origine
7. Clôture	Fiche refermée en référénçant le commit correctif et la date de déploiement

Définition de « corrigé » : correctif déployé en production, comportement d'origine vérifié sur l'environnement réel, test automatisé couvrant le cas lorsque c'est possible, évolution inscrite au journal des versions. Tant que ces quatre conditions ne sont pas réunies, l'anomalie reste ouverte : un correctif fusionné mais non

déployé ne corrige rien pour l'utilisateur.

§4. Fiche de consignation d'une anomalie

Compétence C4.2.1 (éliminatoire) : La fiche de consignation contient les informations permettant de reproduire le bogue ; l'analyse du bogue et les préconisations de correction sont explicitées et permettent de corriger l'anomalie.

4.1 L'anomalie retenue et son identification

L'anomalie traitée ici relève du cas le plus difficile pour un dispositif de maintien en condition opérationnelle : une anomalie **invisible pour la supervision technique**. L'API répondait, ne levait aucune exception, et renvoyait des codes d'erreur conformes à son propre code ; seuls les utilisateurs pouvaient la signaler.

Référence	#67
Titre	Déconnexion à la fermeture du navigateur en production (session non persistée)
Détectée le	17/07/2026
Canal de détection	Retour utilisateur direct (plusieurs utilisateurs), confirmé par reproduction
Environnement	Production : front et API
Supports concernés	Tous : ordinateur, mobile, application installée en PWA
Version	1.3.1
Sévérité	high : fonctionnalité principale bloquée, sans contournement
Étiquettes	bug, production, severite:high

[Bug] Déconnexion à la fermeture du navigateur en production (session non persistée) #67

Closed

Affy657 opened 1mo ago

Fiche consignée à posteriori. L'anomalie a été détectée et corrigée le 17/07/2026, avant la mise en place du processus formalisé de consignation (docs/NKP/bloc-4-mco/processus-anomalies.md). Elle est reconstituée ici à partir du diagnostic d'origine et du correctif déployé, pour servir de référence au processus. Les dates réelles sont indiquées à chaque étape.

Contexte

Détectée le	17/07/2026
Canal de détection	Retour utilisateur direct (plusieurs utilisateurs), confirmé par reproduction
Environnement	Production — web.movie-picker.fr + API Cloud Run
Supports concernés	Tous : desktop, mobile, application installée (PWA)
Version	v1.3.1
Sévérité	high — fonctionnalité principale bloquée (authentification)

Étapes pour reproduire

1. Se connecter à l'application avec un compte existant.

2. Vérifier que la session est active (l'utilisateur est bien identifié).

3. Fermer complètement l'onglet ou le navigateur.

4. Attendre une période d'inactivité suffisante pour que le service Cloud Run redescende à zéro instance (scale-to-zero, quelques dizaines de minutes).

5. Rouvrir l'application.

Comportement attendu

La session persiste : l'utilisateur reste connecté, le cookie d'authentification étant émis avec une durée de vie longue.

Comportement observé

New issue

Assignees

No one - Assign yourself

Assign to Agent

Labels

bug production severite:high

Projects

No projects

Milestone

No milestone

Relationships

None yet

Development

Create a branch for this issue or link a pull request.

Notifications

Unsubscribe

Customize

You're receiving notifications because you're subscribed to this thread.

Participants

Open in GitHub Copilot app

Transfer issue

Clone issue

Fiche de consignation de l'anomalie #67 : contexte, étapes de reproduction et comportements attendu et observé. La fiche se poursuit par l'analyse et les préconisations reprises aux §4.3 et §4.4

4.2 Reproduction

Étapes

1. Se connecter à l'application avec un compte existant.

2. Vérifier que la session est active (l'utilisateur est bien identifié).
3. Fermer complètement l'onglet ou le navigateur.
4. Attendre une période d'inactivité suffisante pour que le service redescende à zéro instance (quelques dizaines de minutes).
5. Rouvrir l'application.

Comportement attendu. La session persiste : l'utilisateur reste connecté, le cookie d'authentification étant émis avec une durée de vie longue.

Comportement observé. L'utilisateur est déconnecté et renvoyé vers l'écran de connexion ; l'API répond **401** sur les appels authentifiés.

Élément déterminant. Le phénomène est apparu **sans aucun changement de code**. Les utilisateurs le formulent ainsi : « avant, ça marchait ». L'étape 4 est celle qui rend la fiche exploitable : sans elle, la reproduction échoue une fois sur deux et l'anomalie passe pour intermittente.

4.3 Analyse

Le diagnostic a mis au jour **deux causes cumulées**, l'une expliquant le déclenchement, l'autre aggravant silencieusement la situation depuis l'origine.

Cause racine 1 : clé de chiffrement expirée. Le cookie d'authentification est chiffré par le mécanisme de protection des données du framework. La clé provenait d'un secret généré **sans durée explicite**, donc avec la valeur par défaut de **90 jours**. Créée lors du déploiement de l'authentification, le 7 avril 2026, elle est arrivée à expiration début juillet. Une clé expirée reste capable de déchiffrer les cookies déjà émis mais ne peut plus en chiffrer de nouveaux : seules les sessions ouvertes après cette date se sont mises à casser, d'où des signalements qui ne remontent qu'à la mi-juillet. Le trousseau ne contenant qu'une seule clé, chaque instance s'est mise à en régénérer une **éphémère**, stockée dans un dossier temporaire effacé au démarrage. Le service étant configuré sans instance minimale, le démarrage à froid suivant rendait le cookie émis par l'instance précédente indéchiffrable : 401, puis déconnexion. Cette cause explique les trois observations : le lien avec la fermeture du navigateur (le temps d'inactivité laisse le service redescendre à zéro), l'atteinte de tous les supports (le défaut est côté serveur), et l'apparition sans déploiement (c'est le temps qui déclenche, pas le code).

Cause racine 2 : configurateur jamais exécuté. Le composant chargé de configurer le cookie était enregistré sur une interface que la fabrique d'options **ne consomme pas**. Il ne s'exécutait donc jamais : le cookie portait le nom par défaut du framework, le magasin de sessions était inactif, les réglages de sécurité inopérants, et un 401 renvoyait une redirection vers une route de connexion inexistante dans cette API. Présent depuis l'origine sans jamais se manifester, ce défaut n'a été révélé que par l'investigation de la première cause. Vérification du diagnostic : un test isolé de l'injection de dépendances échoue avec l'ancien enregistrement.

4.4 Préconisations de correction

#	Préconisation	Effet attendu
1	Persister les clés de chiffrement en base, partagées entre instances et révisions	Supprime la cause 1 : plus de perte de clé au redémarrage
2	Corriger l'enregistrement du configurateur sur l'interface consommée par la fabrique d'options	Supprime la cause 2 : nom du cookie, magasin de sessions et réglages de sécurité effectifs
3	Porter la durée de session à 30 jours glissants, en une constante unique	Aligne la durée réelle sur la promesse faite à l'utilisateur
4	Allonger la durée de la clé de secours hors base	Évite que le repli reproduise l'expiration à 90 jours

#	Préconisation	Effet attendu
5	Ajouter un test de non-régression sur l'application du configurateur	Interdit la réapparition silencieuse de la cause 2

Effet de bord à anticiper : le changement de clé et l'activation du magasin de sessions invalident les cookies existants, imposant une **reconnexion unique** de tous les utilisateurs au déploiement, à annoncer plutôt qu'à subir.

Cette fiche a été consignée **a posteriori** : l'anomalie du 17 juillet 2026 précède la formalisation du processus décrit au §3, et elle a été reconstituée à partir du diagnostic d'origine et du correctif déployé, avec les dates réelles de chaque étape. Le processus s'applique depuis en amont : l'anomalie suivante, détectée le 25 juillet par les portes du pipeline, a été consignée au moment de sa constatation (fiche #68, §5.4).

§5. Traitement d'une anomalie détectée en production

Compétence C4.2.2 : Créer et déployer un correctif en respectant le processus d'intégration et de déploiement continu afin de résoudre l'anomalie. Le traitement tire profit du processus d'intégration et de déploiement continu ; le correctif mis en place est décrit et permet la résolution de l'anomalie.

5.1 Le pipeline mobilisé

Un correctif emprunte le même chemin qu'une évolution : **aucune voie rapide, aucun accès direct à la production**, l'urgence étant précisément le moment où l'on est tenté de sauter les vérifications. Le pipeline se déclenche sur **toute poussée, quelle que soit la branche**, et la compare à la branche principale : les portes s'exécutent avant la fusion, sur le périmètre exact qu'elle apportera, seuls la publication et le déploiement étant réservés à la branche principale.

Étape	Contrôles	Bloquant
Poussée sur une branche	Analyse statique, formatage, compilation, recherche de secrets	✓
Tests	Unitaires front (643) et API (820), intégration (101), 6 parcours de bout en bout	✓
Qualité et sécurité	Porte de qualité sur le code nouveau, vulnérabilités des dépendances et de l'image, audit performance et accessibilité	✓
Fusion	Auto-relecture du diff complet, les portes automatiques compensant l'absence d'un second relecteur	n/a
Construction et publication	Image conteneur analysée puis publiée, référencée par empreinte	✓
Déploiement	Mise en ligne de la révision (API), invalidation du cache (front)	n/a
Contrôle post-déploiement	Appel des deux sondes de santé : répond-il, et peut-il servir ?	✓

Le déploiement n'est donc pas un acte manuel, mais la conséquence d'une fusion ayant franchi l'ensemble des portes. En cas d'incident malgré tout, un retour arrière bascule la totalité du trafic vers la révision précédente **sans reconstruction**, en quelques minutes, ce qui rend praticable l'arbitrage du §3.4.

5.2 Chronologie du traitement de l'anomalie #67

Étape	Contenu
Signalement (17/07)	Plusieurs utilisateurs rapportent des déconnexions ; qualification et reproduction
Analyse	Identification des deux causes racines (§4.3)
Correctif	Branche <code>fix/auth-session-persistence</code> , commit <code>ec7ce77</code> avec test de non-régression
Portes du pipeline	Analyse statique, tests, parcours de bout en bout, qualité, sécurité : toutes franchies
Fusion et relecture	<code>21335d2</code> sur la branche principale, puis ajustements de revue (<code>f80c95b</code>) fusionnés par <code>3a92e28</code> , les treize travaux du pipeline franchis jusqu'aux deux déploiements
Déploiement	Automatique : publication de l'image, mise en ligne de la révision et du front
Contrôle post-déploiement	Sondes de santé appelées par le pipeline, réponses conformes
Vérification fonctionnelle	Session testée après déploiement, puis après un démarrage à froid , condition exacte du défaut
Clôture	Fiche refermée en référénçant le commit correctif et la date de déploiement

5.3 Le correctif mis en place

Les cinq préconisations du §4.4 ont toutes été réalisées. Le dépôt de clés est adossé à la base de données, ce qui les rend durables et partagées entre instances et révisions sans interrompre la rotation automatique, le repli sur fichier restant en développement. Le configurateur est enregistré sur l'interface effectivement consommée par la fabrique d'options, si bien que le nom du cookie applicatif, le magasin de sessions, les réglages de sécurité et la réponse 401 en JSON redeviennent effectifs. La durée de session passe à trente jours glissants par une constante unique partagée par les trois composants concernés, la clé de secours hors base à dix ans, et un test de non-régression échoue avec l'ancien enregistrement.

Pourquoi le correctif résout l'anomalie. La cause racine 1 disparaît parce que la clé n'est plus dans un espace éphémère : ni un redémarrage, ni un passage à zéro instance, ni un déploiement ne la lui font perdre, et le cookie reste déchiffrable. La cause racine 2 disparaît parce que la configuration s'applique enfin, ce qu'un test garantit à chaque exécution du pipeline. La reconnexion unique annoncée s'est produite comme prévu, puis les sessions sont restées stables. **Aucune réapparition depuis le 17 juillet 2026.**

5.4 Un second cas : la porte qui bloque avant l'utilisateur

Le 25 juillet 2026, l'ajout du lien « Signaler un problème » fait échouer **cinq parcours de bout en bout** sur six : le libellé d'accessibilité du nouveau lien contient le mot « e-mail », si bien que le sélecteur `getByLabel('E-mail')` des tests désigne deux éléments. La porte étant bloquante et le déploiement du front en dépendant, la mise en ligne est annulée : **le défaut n'a jamais atteint la production**. Il est consigné en fiche #68 selon le même processus ; rendre le sélecteur exact a été retenu plutôt que dégrader le libellé d'accessibilité. Le pipeline apparaît ainsi aux deux moments du cycle de vie, corrigeant une anomalie parvenue jusqu'aux utilisateurs et empêchant un défaut de les atteindre.

§6. Recommandations argumentées d'amélioration

Compétence C4.3.1 : Proposer des axes d'amélioration en prenant en compte les indicateurs de performance et en analysant les retours utilisateurs, afin de maintenir et renforcer l'attractivité du logiciel.

6.1 Méthode

Les recommandations qui suivent partent de mesures. Cinq sources ont été croisées : la **base de production** (agrégats sans donnée personnelle), les **métriques d'exploitation** sur trente jours, l'**analytique produit** sur quatre-vingt-dix jours, limitée aux pages vues et aux web vitals faute d'événements métier, les **audits automatisés** de chaque déploiement, et les **retours utilisateurs**. Ce dernier volet est le plus récent : le questionnaire, en ligne depuis le 18 août 2026, a recueilli sept réponses sur dix-sept comptes. L'échantillon est réduit et orienté vers les utilisateurs les plus engagés, cinq répondants sur sept utilisant l'application à chaque soirée : ses tendances nuancent les mesures quantitatives sans les fonder.

6.2 Indicateurs observés

Indicateur	Mesure	Lecture
Utilisateurs inscrits	17 (avril à juillet 2026)	Base réduite, usage entre proches
Soirées créées	19 : rythme mensuel 2 / 6 / 6 / 5	Activité stable
Soirées menées jusqu'au tirage	14 sur 19 : 74 %	Le parcours principal aboutit
Films proposés, participations	62 (3,3 par soirée), 80 (4,2 par soirée)	Le partage par lien fonctionne
Votes exprimés	81 (68 pour, 13 contre)	56 % des participations sans aucun vote ; les autres votent 2,3 fois pour 3,3 films
Soirées tirées en mode pondéré	0 sur 19	Le vote n'a jamais influencé un tirage
Films dotés d'une note de présentation	3 sur 62 : 5 %	Fonctionnalité quasi ignorée
Abonnements aux notifications système	3 sur 17 : 18 %	Fonctionnalité peu adoptée
Relations de suivi	21	Fonctionnalité sociale utilisée
Recommandation du produit (0 à 10)	9,6 , aucune note sous 8 (n = 7)	Aucun détracteur, échantillon volontaire
Latence p95, taux d'erreur	207 ms, 0,026 %	Aucun problème de fiabilité
Événements du parcours cœur	0	Création, vote et tirage non instrumentés

Deux conclusions structurent le reste. **Ni la fiabilité ni l'appréciation générale ne sont le facteur limitant**, aucun indicateur technique n'étant dans le rouge et la note de recommandation ne relevant aucun détracteur. **L'engagement dans la soirée l'est** : le vote, mécanisme censé faire émerger le consensus, est à peine sollicité, et plusieurs fonctionnalités livrées ne rencontrent pas leur usage.

6.3 Recommandations

R1. Instrumenter le parcours cœur. *Priorité 1, prérequis des autres.* Aucun événement n'est capturé sur la création d'une soirée, l'ajout d'un film, le vote ou le tirage. Les chiffres du §6.2, reconstitués depuis la base, décrivent des résultats et jamais des abandons : ils ne permettent pas de répondre à « combien d'invités ouvrent le lien sans jamais voter ? ». *Proposition* : capturer six événements et construire l'entonnoir correspondant, l'infrastructure analytique existant déjà. *Coût 0,5 à 1 jour, délai immédiat.* *Gain* : mesure des abandons étape par étape, les décisions suivantes cessent d'être des paris.

R2. Réconcilier le vote et son effet sur le tirage. *Priorité 2. 56 % des participations n'ont produit aucun vote.* La cause tient moins à l'intensité du vote qu'à son absence d'effet : la roue accepte un tirage strictement aléatoire et un tirage pondéré par les votes, le premier est la valeur par défaut, et **aucune des 19 soirées n'a activé le second**. Le produit demande un effort dont il n'utilise pas le résultat. Le questionnaire le confirme : un répondant relance manuellement la roue jusqu'à un résultat qui convienne à tous, alors que le réglage est connu de cinq répondants sur sept. La barrière n'est pas sa découvrabilité mais son statut par défaut. Trois répondants sur sept souhaiteraient même que le vote élimine les films rejetés, piste notée mais non retenue, l'échantillon étant trop réduit pour l'engager. *Proposition*, en trois volets : faire du mode pondéré la valeur par défaut, l'hôte restant libre de revenir à l'aléatoire ; afficher sur la roue la part réelle de chaque film ; signaler à l'hôte la proportion de participants n'ayant pas voté. *Coût 2 à 3 jours, délai d'une itération après R1. Gain : le vote retrouve la fonction qui justifie sa présence, faire émerger un consensus, promesse même du produit. Objectifs : la moitié des soirées tirées en mode pondéré, participations sans vote sous 25 %.*

R3. Rendre les notifications atteignables avant de trancher leur sort. *Priorité 3.* Trois abonnements actifs pour dix-sept inscrits, alors que la 1.1 a livré les clés de signature, cinq déclencheurs et une interface de préférences. Le code explique le chiffre : l'activation n'est **jamais proposée dans le parcours**, elle n'existe que sous la forme d'un interrupteur que rien ne signale dans « Mon compte », et le réglage par type ne s'affiche qu'une fois l'utilisateur abonné. Le taux ne mesure donc pas un refus mais une absence de sollicitation, ce que le questionnaire confirme : quatre répondants sur sept ignoraient que l'activation était possible, et l'un d'eux demande spontanément à être averti quand un film est ajouté à une soirée qu'il a rejointe, notification qui existe déjà mais reste invisible. *Proposition* : proposer l'activation une fois, juste après avoir créé ou rejoint une soirée ; rendre le choix par type visible avant l'abonnement. *Coût 1 jour, délai d'une itération. Gain : une décision fondée. Si l'adoption ne dépasse pas 40 % en deux mois, le gel devient défendable ; aujourd'hui il condamnerait une fonctionnalité que personne n'a eu l'occasion d'accepter.*

R4. Installer une boucle de satisfaction continue. *Priorité 3.* Aucun dispositif ne mesure la satisfaction dans la durée : le questionnaire donne une photographie, pas une tendance, et sa note de 9,6 ne dira rien d'une dégradation à venir. *Proposition* : une question unique après le tirage, « cette soirée s'est-elle bien passée ? » à trois niveaux, stockée sans donnée nominative et agrégée par mois. *Coût 1 à 2 jours, délai d'une itération. Gain : détection des dégradations d'expérience invisibles pour la supervision technique, aucune des anomalies fonctionnelles rencontrées n'ayant levé d'exception.*

R5. Mesurer le pari de la récurrence plutôt que l'étendre. *Priorité 4, conditionnée à R1.* Dix-neuf soirées en trois mois et demi pour dix-sept inscrits : l'application est utilisée par événement, pas par habitude. La 1.4 en préparation mise sur la récurrence, avec la sélection manuelle et la flamme de régularité, sans qu'aucune mesure n'éclaire ce pari ; le questionnaire va dans l'autre sens, cinq répondants sur sept déclarant que rien ne les ferait revenir plus souvent. *Proposition* : ne pas prolonger cet axe avant d'avoir observé l'effet réel de la flamme sur l'entonnoir de R1, puis retenir d'abord la reconduction d'une soirée avec le même groupe en un clic. *Coût 2 jours contre 5 à 8 jours pour une gamification étendue, délai après un mois d'observation de R1. Gain : évite d'engager une semaine sur une hypothèse non vérifiée, et supprime le principal frein à une nouvelle soirée, reconstituer le groupe.*

R6. Corriger deux irritants remontés par le questionnaire. *Priorité 4.* Le texte libre a fait remonter deux frictions absentes des mesures quantitatives : un répondant doit se reconnecter à chaque ouverture du lien de soirée depuis le navigateur intégré de Snapchat, symptôme distinct de l'anomalie #67 et consigné en fiche #71 ; un autre signale que le profil public n'affiche que le nombre de films proposés, jamais lesquels. *Proposition* : détecter les navigateurs intégrés connus pour suggérer l'ouverture dans le navigateur système ; lister les films proposés sur le profil. *Coût 1 jour, délai immédiat. Gain : deux frictions concrètes levées, remontées par deux répondants différents.*

6.4 Priorisation et périmètre

Rang	Recommandation	Coût	Délai de mise en œuvre
1	R1, Instrumenter le parcours cœur	0,5 à 1 j	Immédiat
2	R2, Réconcilier le vote et son effet	2 à 3 j	Une itération après R1
3	R3, Rendre les notifications atteignables	1 j	Une itération
3	R4, Boucle de satisfaction	1 à 2 j	Une itération
4	R5, Récurrence (option courte)	2 j	Après un mois d'observation de R1
4	R6, Deux irritants du questionnaire	1 j	Immédiat

Total : 7,5 à 10 jours, en quatre lots. Aucune recommandation n'exige de refonte, toutes s'appuyant sur l'existant, condition de leur faisabilité à une seule personne ; l'ordre n'est pas une file d'attente, R1 conditionnant l'évaluation de R2, R3 et R5. La fiabilité et la performance sont délibérément absentes : elles ne limitent pas l'attractivité du produit.

§7. Journal des versions déployées

Compétence C4.3.2 (éliminatoire) : *Établir un journal des versions déployées en y intégrant la documentation des correctifs réalisés, pour suivre les différentes évolutions du logiciel. Le journal contient les améliorations apportées par la version ; les correctifs déployés sont documentés.*

7.1 Dispositif

Le journal repose sur trois supports complémentaires, tous versionnés ou publiés.

Support	Rôle
CHANGELOG.md , à la racine du dépôt	Journal de référence : format <i>Keep a Changelog 1.1.0</i> , rubriques canoniques du format (<i>Added, Changed, Removed, Fixed, Security</i>), contenu en français
Étiquettes Git	Une étiquette annotée vX.Y.Z par version, posée sur le commit exact déployé
Publications GitHub	Notes de version en français, adossées à l'étiquette, mêmes rubriques traduites

La version est aussi **lisible depuis l'application**, affichée en pied de page et exposée par la sonde d'aptitude à servir. Un utilisateur qui signale une anomalie transmet donc sa version sans avoir à la chercher, le message pré-rempli l'embarquant automatiquement.

7.2 Politique de versionnage

Versionnage sémantique, interprété ainsi pour une application web : **majeur** pour une rupture du parcours ou du contrat de l'API, aucun à ce jour ; **mineur** pour un nouveau parcours ou écran, comme la 1.2.0 et son profil public ; **correctif** pour une correction d'anomalie, un point de sécurité, une amélioration d'exploitation ou l'enrichissement d'un parcours existant, comme la 1.3.2. Le critère du mineur est l'ouverture d'un parcours, pas la visibilité d'un changement, ce qui évite l'inflation du numéro mineur sur un produit livré en continu. Les entrées décrivent le produit, jamais les commits.

7.3 Versions publiées

Version	Date	Contenu principal
1.3.2	25/07/2026	Supervision de production, canal de signalement, suivi des erreurs, SEO, correctif des sessions, portes de qualité bloquantes
1.3.1	08/07/2026	Refonte des cartes film, politique de sécurité du contenu, refonte du pipeline, durcissement SSRF, lacunes de tests comblées
1.3.0	19/06/2026	Roue animée, recherche avancée, offres de location, export calendrier, états vides, refonte de la navigation
1.2.0	11/06/2026	Profil public, suivi entre utilisateurs, notifications dans l'application, accessibilité étendue, export et suppression RGPD
1.1.0	25/05/2026	Application installable, notifications système à cinq déclencheurs, séries, bandes-annonces, plateformes
1.0.0	19/05/2026	Premier périmètre de production consolidé, déployé progressivement depuis avril : comptes, soirée partageable, propositions, votes, roue, temps réel
0.1.0	27/02/2026	Prototype : création de soirée, proposition de films, vote, roue de tirage

Les dates sont celles de la mise en production effective. Le versionnage ayant été formalisé le 8 juillet 2026, les cinq versions antérieures ont été étiquetées ce jour-là, chaque étiquette sur le commit réellement déployé à l'époque ; depuis la 1.3.1, publiée le même jour, l'étiquette est posée au moment de la mise en production.

7.4 Exemple : la note de version 1.3.2

Version d'exploitation : elle met l'application sous supervision active et ouvre un canal de signalement aux utilisateurs.

Releases Tags Draft a new release Q Find a release

Release list

- v1.3.2 - Supervision d...
- v1.3.1 - Durcissement ...
- v1.3.0 - États vides, ex...
- v1.2.0 - Social, accessi...
- v1.1.0 - PWA, notificat...
- v1.0.0 - Première versi...
- v0.1.0 - Prototype initi...

v1.3.2 - Supervision de production et canal de support Latest Compare

Affy657 released this last month v1.3.2 685c978

Version d'exploitation : elle met l'application sous supervision active et ouvre un canal de signalement aux utilisateurs.

Ajouté

- Lien « Signaler un problème » en pied de page : ouvre un message pré-rempli (description, étapes, comportement attendu et observé) avec le contexte technique (page, version, navigateur).
- Sonde de disponibilité applicative `GET /health/ready` : vérifie la joignabilité de MongoDB, renvoie 503 si la base est injoignable, et expose la version déployée.
- Supervision de production : trois sondes de disponibilité (API, readiness, front) interrogées depuis trois continents, cinq politiques d'alerte (indisponibilité, base injoignable, erreurs serveur, latence dégradée) notifiées par e-mail, tableau de bord d'exploitation, et alertes Sentry sur les régressions et les rafales d'erreurs.
- Vérification de la readiness dans le **smoke test de déploiement** : une révision dont la base est injoignable fait échouer sa propre mise en production.
- Monitoring d'erreurs Sentry sur le front (React) et l'API (.NET), en production uniquement, avec une catégorie « surveillance des erreurs » dans les préférences de confidentialité.
- SEO global : métadonnées par page, image Open Graph, données structurées JSON-LD et sitemap dynamique des profils publics.
- QR code de partage sur le profil public.
- Filtre de durée de film dans la recherche d'ajout (10 min ou moins à 3h et plus).
- Réglage de compte pour l'échelle de notes TMDB (affichage sur 5 ou sur 10).
- Nouveau logo (clap incliné, fond sombre) et icônes PWA régénérées.

Note de version 1.3.2 publiée, adossée à son étiquette et au commit déployé, la liste des sept versions apparaissant à gauche

Ajouté

- Lien « Signaler un problème » en pied de page, ouvrant un message pré-rempli avec la page, la version et le navigateur.

- Sonde `GET /health/ready` vérifiant la joignabilité de la base et exposant la version déployée, jouée en fin de déploiement.
- Supervision de production : trois sondes, cinq politiques d'alerte notifiées par courriel, tableau de bord, règles sur les régressions et les rafales d'erreurs.
- Suivi des erreurs sur le front et l'API, avec sa catégorie dans les préférences de confidentialité.
- Référencement (métadonnées, image de partage, données structurées, plan de site), filtre de durée, échelle de notes au choix, code QR de profil, version en pied de page.

Modifié

- Portes de qualité du pipeline rendues bloquantes (qualité du code, performance, parcours de bout en bout).
- Performance et accessibilité : accessibilité 100, décalage de mise en page éliminé, poids du paquet réduit de 83 %.
- Duplication de code réduite : actions sur un film, fermeture des modales, pied de carte partagé.

Retiré

- Barre de couleur du thème sur la page de détail d'une soirée.

Corrigé

- **Sessions non persistées en production (fiche #67)** : les utilisateurs étaient déconnectés à la fermeture du navigateur, sans changement de code. Clés de chiffrement désormais persistées en base et partagées entre instances, configurateur de cookie enregistré sur l'interface effectivement consommée, durée de session unifiée à trente jours glissants, test de non-régression ajouté. Une reconnexion unique a été nécessaire au déploiement.
- Cookie de session passé en `SameSite=Lax` en production.
- Images cassées en production : la politique de sécurité du contenu bloquait les affiches et les avatars par défaut.
- Sept signalements de qualité du code résolus ; envoi des courriels de production rebranché ; correction du saut transitif du pipeline qui empêchait les deux travaux de déploiement de s'exécuter.
- Les points d'entrée de lancement et de clôture de la roue exigent un corps JSON, alignés sur le reste de l'API ; l'animation de la roue respecte la préférence système de réduction des animations.

Sécurité

- Montée du routeur de 7.18.1 vers 8.3.0, corrigeant l'avis `GHSA-qwww-vcr4-c8h2`.
- Résolution des huit alertes de dépendances ouvertes (six hautes, deux basses).
- Remplacement de l'audit natif par Trivy, le service amont ayant été retiré : l'audit ne scannait plus aucun fichier depuis son introduction.
- Déblocage du pipeline : vulnérabilité de l'image Docker de base et version de Java obsolète pour l'analyseur de qualité.

7.5 Traçabilité des correctifs

De l'anomalie vers la version : la fiche référence le commit correctif, le commit appartient à une étiquette, l'étiquette correspond à une entrée du journal ; l'anomalie #67 se retrouve ainsi dans la version qui la corrige. **De la version vers les anomalies** : la rubrique *corrigé* énumère les correctifs embarqués, la rubrique *sécurité* les vulnérabilités traitées avec leur identifiant d'avis public. **De la production vers le code** : la version portée par chaque événement d'erreur, égale à l'identifiant du commit déployé, rattache une exception observée en production au déploiement qui l'a introduite.

§8. Problème résolu en collaboration avec le support

Compétence C4.3.3 : Collaborer avec les équipes de support en fournissant une expertise technique, en répondant aux retours clients et en résolvant des problèmes complexes afin d'améliorer le logiciel.

8.1 Dispositif de support

L'application est développée et exploitée par une seule personne : les rôles de support de premier niveau (réception et qualification) et de second niveau (diagnostic et correction) sont tenus par le même intervenant. Le dispositif est donc conçu pour que le retour utilisateur ne dépende pas d'un canal informel : la réception passe par le lien « **Signaler un problème** » en pied de page, dont le message pré-rempli embarque la page, la version et le navigateur ; la qualification suit le gabarit et la grille de sévérité du §3 ; le diagnostic s'appuie sur les outils du §2 ; le retour à la normale est confirmé en production avec les utilisateurs signalants. La faiblesse structurelle est connue, signalant et correcteur ne se contrôlant pas : elle est compensée par la formalisation écrite de chaque anomalie.

8.2 Le contexte du retour utilisateur

L'anomalie traitée aux §4 et §5 illustre ce dispositif, en tant qu'elle a été **détectée par les utilisateurs et par eux seuls**. Le 17 juillet 2026, plusieurs d'entre eux signalent le même symptôme en langage courant : « *je dois me reconnecter à chaque fois* ». Les échanges de qualification apportent trois précisions qu'aucun outil n'aurait fournies :

1. La déconnexion survient à la **fermeture de l'onglet ou du navigateur**, pas pendant l'utilisation.
2. Elle touche **tous les supports** : ordinateur, mobile, application installée.
3. « **Avant, ça marchait** », alors qu'aucune version n'a été publiée entretemps.

Traduit en termes techniques, le problème à résoudre devient : pourquoi un cookie émis avec une durée de vie longue cesse-t-il d'être reconnu après la fermeture du navigateur, sans modification du code ?

8.3 La résolution apportée

La troisième précision a écarté d'emblée l'hypothèse d'une régression de code et orienté vers un mécanisme dépendant du temps. L'investigation a mis au jour deux causes cumulées : une clé de chiffrement arrivée à expiration, combinée à un stockage éphémère et au passage à zéro instance du service, et un composant de configuration enregistré sur une interface jamais consommée, inopérant depuis l'origine. La première a été supprimée en persistant les clés en base, la seconde en corrigeant l'enregistrement du composant ; le détail figure aux §4.3 et §5.3.

Un point relevait de la relation avec les utilisateurs plutôt que de la technique : le changement de clé imposait une **reconnexion unique pour tous**, annoncée avant le déploiement. Un utilisateur prévenu y voit une opération de maintenance ; non prévenu, il y voit une seconde anomalie. L'épisode a par ailleurs fait naître le canal de signalement en pied de page, qui embarque d'emblée les trois informations qu'il avait fallu réclamer.

8.4 Contribution des parties prenantes

Partie prenante	Contribution	Sans elle
Utilisateurs signalants	Détection, description du symptôme, les trois précisions de contexte	L'anomalie restait invisible : aucune alerte, aucune exception

Partie prenante	Contribution	Sans elle
Développeur-mainteneur	Qualification, reproduction, diagnostic, correctif, test, déploiement, vérification	n/a
Hébergeur de l'API	Le passage à zéro instance documenté, chaînon entre l'expiration de la clé et le symptôme	Le lien restait incompréhensible
Base de données managée	Persistance durable des clés entre instances et révisions	La correction se limitait à repousser l'expiration
Pipeline d'intégration et de déploiement	Portes de qualité, déploiement, contrôle post-déploiement	Correctif livré sans garantie de non-régression

La contribution la plus déterminante n'est pas technique : c'est le « **avant, ça marchait** », qui a exclu la piste la plus naturelle et orienté vers l'expiration d'une clé. Un signalement limité à « je suis déconnecté » aurait coûté plusieurs heures de recherche, probablement dans la mauvaise direction. Sur un projet à intervenant unique, la collaboration avec le support se joue ainsi entre le développeur et ses utilisateurs, ceux-ci tenant le rôle de détection et de qualification qu'assurerait un support de premier niveau.